

黄佳 - ClaudeCode实践 - 第7章 无为而治：Headless模式与CI-CD集成

运筹帷幄之中，决胜于千里之外。

公司的技术团队仅有3名高级工程师，却每天需要审查二十多个PR。按照惯例，每个PR必须至少获得一名资深工程师的批准(Approved)方可合并。后果显而易见：PR大量积压，发布频率被迫降低；开发者在等待审查期间不得不切换至其他任务，导致上下文频繁中断，效率受损。更为严峻的是，在审查负荷最高的时段，人工审查的质量开始下滑——疲惫的审查者极易遗漏安全漏洞或性能隐患，而这些问题一旦流入生产环境，其修复代价将成倍增加。

经过一周的调研，小雪提出了一套解决方案：引入Claude作为PR的“初审员”，自动执行安全检查、代码质量分析及测试覆盖率评估，并生成结构化的审查报告。人工审查员则基于该报告进行最终决策，将精力集中于业务逻辑与架构设计等核心领域。该方案并非替代人工审查，而是将工程师从重复性的机械检查中解放出来，使其专注于唯有才能胜任的高阶判断。

“Claude Code能在无人值守的情况下自动运行吗？”小冰问。

“这正是Headless模式的核心。”咖哥解释道，“在前面6章中，我们一直是在‘对话模式’下使用Claude——你输入指令，它给出反馈，如此往复。但从本章开始，Claude Code将脱离人类的实时操作，能够在后台、在CI/CD流水线中，甚至在凌晨三点无人值守时独立工作。这种转变的意义，堪比Docker的出现将应用从‘绑定于特定机器’解放为‘可在任何环境中运行’。”

7.1 从人机交互到无人值守：一次关键的架构演进

在交互模式下，Claude Code扮演着“在场对话伙伴”的角色：用户输入指令，Claude即时响应，待用户确认结果后继续下一轮交互。这种模式非常适合于探索性工作，如代码调试、架构方案的头脑风暴或模块的渐进式重构。然而，在自动化场景中，键盘前无人守候，亦无人为中间步骤把关，整个流程必须具备在零人工干预下从头至尾独立运行的能力。

Headless模式正是为此所设计的。通过-p（或--print）参数，用户可明确告知Claude Code：“这是一次非交互式调用，请在任务完成后直接输出结果并退出，不需要等待任何用户输入。”

```
# 最基本的Headless模式调用
claude -p "分析src/目录的安全漏洞"

# 等价的完整形式
claude --print "分析src/目录的安全漏洞"
```

然而，-p仅仅是入口。真正将Claude Code转化为“可编程组件”的，是其完备的参数体系。要深入理解这套体系，首先需要厘清交互模式与Headless模式的核心差异（见表7-1）。

表7-1 交互模式与Headless模式的核心差异

维度	用户界面	输入方式	输出方式	权限确认	会话持久化	成本控制	典型场景
交互模式	终端TUI，实时渲染	多轮实时对话	流式渲染Markdown	弹出交互提示，需要人工确认	自动保存，支持断点续传(resume)	依赖人工观察与中断	开发调试、架构探索
Headless模式	无界面，仅标准输出(stdout)	一次性Prompt (或stdin管道)	纯文本/JSON/流式JSON	自动跳过或依据白名单执行	默认不保存 (可按需指定)	通过--max-turns/--max-budget-usd硬性限制	CI/CD流水线、定时任务、批处理

这一演进的意义远不止“少了一个界面”那么简单。在交互模式下，人类实际扮演着三重关键角色：权限审批者（实时决策是否允许执行特定命令）、质量检查者（即时判断输出结果是否符合预期）和流程控制者（根据中间结果动态调整下一步指令）。当这3个角色全部缺席时，系统必须在运行前将所有规则“硬编码”到参数中：哪些工具被授权调用？最多执行多少轮对话？成本预算上限是多少？输出必须遵循何种格式以便下游程序解析？这正是Headless模式庞大参数体系的核心使命——将原本依赖人类直觉和实时判断的模糊过程，转化为一套确定性、可预测、可自动执行的工程规范。

7.2 核心参数体系：4个维度的控制

咖哥在白板上勾勒出一个清晰的四象限图，展示了Headless模式参数体系的4个维度，如图7-1所示。

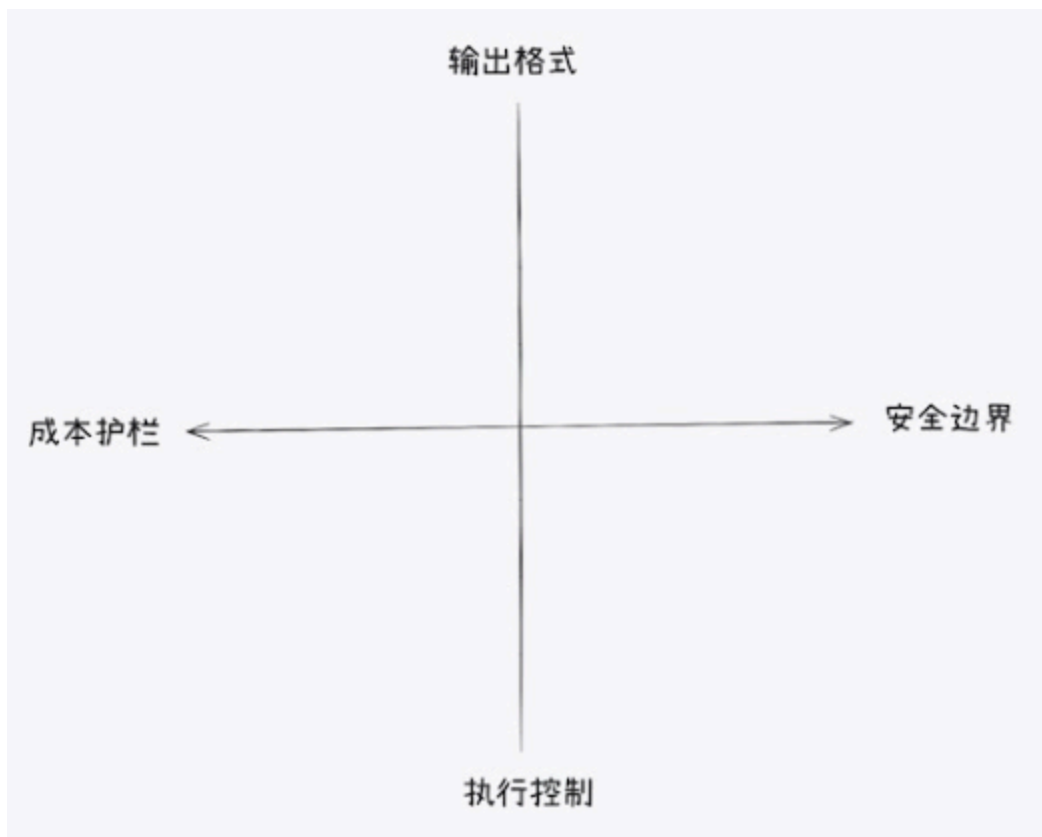


图7-1 Headless模式参数体系的4个维度

“看，”咖哥指着白板说道，“Headless模式的参数并非杂乱无章，而是可以归纳为这4个维度。任何一次生产级的无人值守调用，都必须在这4个维度上作出明确且坚定的决策。”

7.2.1 输出格式控制

`--output-format` 参数是连接Claude Code与下游自动化流水线的“协议转换器”。它明确了输出数据的结构，直接决定了后续脚本或系统如何消费这些结果。

该参数包含3种核心格式——text格式、json格式和stream-json格式。

1. text格式

最简单的纯文本输出，保留了自然的语言流，适用于直接阅读或简单的脚本处理。

```
claude -p "列出主要问题" --output-format text
```

2. json格式

输出为一个完整的、严格结构化的JSON对象。它不仅包含任务结果，还封装了丰富的执行元数据，是构建健壮CI/CD流水线的基石。

```
claude -p "审查 PR" --output-format json
```

当使用json格式时，返回的对象包含审计和监控所需的全量信息。

```
{
  "type": "result",
  "subtype": "success",
  "session_id": "550e8400-e29b-41d4-a716-446655440000",
  "is_error": false,
  "duration_ms": 12345,
  "duration_api_ms": 10000,
  "num_turns": 5,
  "total_cost_usd": 0.0342,
  "usage": {
    "input_tokens": 5000,
    "output_tokens": 1500,
    "cache_creation_input_tokens": 0,
    "cache_read_input_tokens": 3000
  },
  "result": "发现2个 Critical问题: \n1. SQL注入风险...",
  "structured_output": null
}
```

在自动化运维和成本优化中，以下几个字段至关重要。

- `total_cost_usd` 可用于精确成本追踪，在CI/CD流水线中，用户可以将每次运行的成本记录到数据库，生成“每个PR的AI审查成本”报表，甚至对超出预算的部门进行自动告警或阻断。
- `num_turns` 可用于执行轮次监控，反映模型实际交互的深度（对比设定的 `--max-turns` 限制）。
- `usage.cache_read_input_tokens` 反映Prompt缓存的生效情况。在频繁调用相同System Prompt或基础代码库的场景中，`cache_read_input_tokens` 越高，意味着跳过的计算越多，成本越低且速度越快。

3. stream-json格式

逐行输出JSON事件（即NDJSON格式），专为长任务的实时监控而设计。示例如下。

```
claude -p "分析代码" --output-format stream-json
```

输出中，每一行均为一个独立的JSON对象，并严格按时间顺序排列。

```
{"type":"system","subtype":"init","session_id":"...","model":"claude-sonnet-4-6","tools":["Read","Grep","Glob"]}
{"type":"assistant","message":{"role":"assistant","content":
[{"type":"text","text":"正在分析..."}]}}
```

```
{
  "type": "user", "message": {
    "role": "user", "content": [
      {
        "type": "tool_result", "tool_use_id": "...", "content": "..."
      }
    ]
  },
  "type": "assistant", "message": {
    "role": "assistant", "content": [
      {
        "type": "text", "text": "分析完成。"
      }
    ]
  },
  "type": "result", "subtype": "success", "total_cost_usd": 0.03, "result": "最终结果"
}
```

请特别关注首行的system/init事件，它汇报了当前会话的模型版本、可用工具列表及MCP服务器状态等初始化信息。这一特性在调试CI配置问题时非常有用。

“3种格式的选择其实很简单，”咖哥总结道，“需要人工阅读就采用纯文本格式，需要机器解析就采用JSON格式，需要实时监控就采用stream-json格式。”

7.2.2 成本护栏

在交互模式下，你可以随时按下Ctrl+C组合键中断Claude的执行；但在Headless模式下，由于缺乏这种“人工熔断”机制，必须通过参数来实施控制。请看以下代码示例。

```
# 限制执行轮数(防止过度分析)
claude -p "审查代码" --max-turns 5

# 设置硬性成本上限(美元)
claude -p "分析日志" --max-budget-usd 0.50

# 双重保险：同时限制轮数与成本
claude -p "分析整个代码库" --max-turns 10 --max-budget-usd 2.00
```

`--max-turns` 与 `--max-budget-usd` 分别从不同维度进行约束。

- `--max-turns`：限制Claude与工具之间的交互轮数。每调用一次工具计为一轮。例如，一个读取了5个文件后给出结论的任务，通常只需要6轮。
- `--max-budget-usd`：限制API调用的总金额。即使交互轮数很少（如仅3轮），如果每轮涉及大量Token（如读取巨型日志文件），累计成本仍可能很高，此时该参数将起到关键的保护作用。

两者在触发时的行为机制截然不同。

- 当 `--max-turns` 耗尽时，Claude会在当前轮次输出其已得出的结论。尽管该结论可能尚不完整，但通常仍具备参考价值。在JSON输出中，此类事件的subtype标记为 `error_max_turns`。
- 当 `--max-budget-usd` 触发时，执行过程会立即强制停止，不再输出任何后续结论。在JSON输出中，此类事件的subtype标记为 `error_max_budget_usd`。

“在生产环境中，务必同时配置这两个参数。”咖哥强调：

- `--max-turns` 用于防止Claude在处理复杂任务时陷入无限递归，
- `--max-budget-usd` 则能避免因一次意外的大范围分析而耗尽API预算。这两道防线缺一不可。

7.2.3 安全边界：工具权限

在Headless模式下，由于缺乏人工确认工具调用的环节，必须预先划定Claude的能力边界，这是CI/CD场景中最关键的安全设置。请看以下代码示例。

```
# 白名单模式：仅允许只读工具(最安全的审查配置)
claude -p "审查代码变更" --allowedTools "Read,Grep,Glob"

# 黑名单模式：禁止特定工具
claude -p "生成文档" --disallowedTools "Bash"

# 细粒度模式：允许特定的命令模式
claude -p "检查 Git 历史" --allowedTools "Read,Grep,Glob,Bash(git log
*),Bash(git diff *)"
```

`--allowedTools`（白名单）与 `--disallowedTools`（黑名单）在安全语义上截然不同。

- **白名单基于“封闭世界假设”**：仅允许使用明确列出的工具，其余工具一律拒绝。
- **黑名单基于“开放世界假设”**：除明确禁止的工具以外，其余所有工具均可使用。

在自动化场景中，白名单的安全性永远高于黑名单。这是因为采用白名单时，你不需要预见所有潜在的危险工具，只需要列出任何确实需要的工具即可。

请注意，`--allowedTools` 支持模式匹配语法。例如，`Bash(git log *)` 表示“允许使用Bash工具，但仅限执行以git log开头的命令”。这种细粒度的控制机制，在需要部分Shell能力（如Git操作）却又不敢赋予完整 Bash 权限的场景下尤为实用。

此外，还有一个更为激进的参数——`--dangerously-skip-permissions`，用于跳过所有权限检查。切勿忽视其名称中的dangerously一词，这绝非装饰。在CI环境中，严禁使用此参数，除非你在严格隔离的容器中运行，且确信Claude的操作无法造成超出容器范围的影响。

7.2.4 执行控制：模型 Prompt 结构化输出

我们可以通过模型选择、System Prompt定制、结构化输出来控制Claude的“思考方式”。

1. 模型选择

不同任务适配不同模型，根据需求权衡。请看以下代码示例。

```
# 简单格式检查(快速且低成本)
claude -p "检查代码格式" --model claude-haiku-4-5

# 深度安全审查(使用最强模型)
claude -p "安全漏洞分析" --model claude-sonnet-4-6

# 指定备选模型(主模型过载时自动降级)
claude -p "代码审查" --model claude-sonnet-4-6 --fallback-model haiku
```

`--fallback-model` 是生产中至关重要但极易被忽视的参数。在API高峰期，若主模型不可用导致CI流水阻塞，所有PR都将停滞。设置降级模型可确保流水线在牺牲少许质量的前提下依然保持运行，避免流程中断。

2. System Prompt定制

通过定制System Prompt来定义Claude的角色与行为边界。请看以下代码示例。

```
# 完全替换System Prompt (这将使Claude忘记其默认的代码助手身份，仅遵循新指令)
claude -p "分析代码" --system-prompt "你是一名专注于OWASP Top 10的应用安全专家。"

# 追加System Prompt (推荐，在保留Claude基础能力的同时，增加特定约束)
claude -p "审查 PR" --append-system-prompt "务必检查敏感信息硬编码。"

# 从文件加载追加Prompt (适用于包含复杂审查规范的大型项目)
claude -p "审查代码" --append-system-prompt-file ./review-guidelines.txt
```

参数 `--system-prompt` 与 `--append-system-prompt` 的核心区别在于它们对Claude默认行为的处理方式不同。

- `--system-prompt`：彻底覆盖Claude默认的System Prompt。Claude将不再知道自己是一个代码助手，也不再理解如何正确使用Read、Grep、Bash等内置工具。它只会机械地遵循你提供的新指令，极易导致工具调用失败或行为不可控。
- `--append-system-prompt`：在 Claude 默认的 System Prompt之后追加你的自定义内容。Claude保留了所有基础能力，同时严格遵循你添加的额外指示。适用于绝大多数CI/CD自动化场景。

3. 结构化输出

强制Claude输出符合特定JSON Schema的数据，从而实现程序化的无缝集成。请看以下代码示例。

```
claude -p "提取代码中所有安全问题" \
  --output-format json \
```

```
--json-schema '{
  "type": "object",
  "properties": {
    "issues": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "severity": {"type": "string", "enum": ["critical", "high",
"medium", "low"]},
          "file": {"type": "string"},
          "line": {"type": "number"},
          "description": {"type": "string"}
        },
        "required": ["severity", "file", "description"]
      }
    }
  },
  "required": ["issues"]
}'
```

使用 `--json-schema` 参数后，系统会自动验证Claude的输出。如果生成的内容不符合定义的Schema，Claude会自动重试生成过程，直到产出合法有效数据为止。最终验证通过的结构化数据将出现在JSON输出的 `structured_output` 字段中。该功能在需要程序化处理Claude输出的场景中极为有用——不需要编写复杂的正则表达式或启发式规则来解析自由文本，即可直接获取标准的JSON对象，供下游脚本、数据库或报警系统调用。

7.3 Unix管道：将Claude融入命令行 workflow

“我注意到了有一个有趣的设计，”小雪说，“Claude Code似乎支持标准输入(stdin)？”

“没错，”咖哥回答道，“这正是Headless 模式最优雅的特性之一——Claude Code能够无缝接入Unix管道，成为你工具链中灵活的一环。”

Unix哲学的核心是“每个工具只做好一件事，并通过管道将它们组合起来”。Claude Code完全契合了这一设计理念。请看以下代码示例。

```
# 分析日志文件
cat server.log | claude -p "找出所有500错误并总结根本原因"

# 分析git变更
git diff HEAD~1 | claude -p "总结这次提交的变更，并按Conventional Commits规范格式化"
```



```
# 解析API响应
curl -s https://api.example.com/health | claude -p "判断服务是否健康"
```

更有趣的是多段管道组合——Claude既能接收上游数据，也能将处理结果传递给下游工具。请看以下代码示例。

```
# Claude分析 → jq提取 → 发送通知
claude -p "检查是否存在安全漏洞" --output-format json | \
jq -r '.result' | \
mail -s "安全扫描报告" security@company.com

# Grep预过滤 → Claude深度分析
grep -r "TODO" src/ | claude -p "将这些待办事项按优先级分类"

# Claude 生成代码 → 直接写入文件
claude -p "生成一个Express健康检查路由" --output-format text > routes/health.js
```

在批量处理场景中，这种管道组合能力使Claude化身为一个“智能过滤器”。传统的文本处理工具（如Grep、Awk、Sed）擅长基于模式的精确匹配，却缺乏语义理解能力；而Claude虽精通语义分析，但高效处理大规模数据并非其强项。通过管道将两者结合，可实现优势互补：先利用Grep筛选出关键内容，再交由Claude进行深度的语义分析。

7.4 实战一：GitHub Actions自动代码审查

理论阐述至此已经足够，接下来通过一个完整的工程实例进行演示。

Anthropic提供了两种GitHub Actions集成方案：一是采用官方Action (anthropics/claude-code-action@v1)，其封装程度高、配置简便；二是直接调用CLI，其灵活性更强、控制更为精细。

7.4.1 采用官方Action

Claude Code内置了便捷的安装命令：在交互模式下执行 /install-github-app ，即可引导用户完成GitHub App安装、Secret配置及Workflow文件创建的全流程。

若选择手动配置，步骤亦不烦琐。首先，进入GitHub仓库，然后单击

Settings→**Secrets**→**Actions**命令，添加 ANTHROPIC_API_KEY ，随后创建如下Workflow文件。

```
# .github/workflows/claude-review.yml
name: Claude PR Review

on:
  pull_request:
    types: [opened, synchronize, reopened]
  issue_comment:
```

```

    types: [created]

# 针对同一PR的新提交自动取消正在运行的旧任务，以节省资源
concurrency:
  group: ${{ github.workflow }}-${{ github.event.pull_request.number }}
  cancel-in-progress: true

jobs:
  review:
    runs-on: ubuntu-latest
    # 仅在触发PR相关事件或评论中包含"@claude"时运行
    if: |
      github.event_name == 'pull_request' ||
      contains(github.event.comment.body, '@claude')

    permissions:
      contents: read
      pull-requests: write
      issues: write

    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0 # 拉取完整历史，以便进行差异分析

      - uses: anthropics/claude-code-action@v1
        with:
          anthropic_api_key: ${{ secrets.ANTHROPIC_API_KEY }}
          prompt: |
            请审查此PR的代码变更，重点检查以下内容。
            1. 安全漏洞(如SQL注入、XSS攻击、敏感信息硬编码等)。
            2. 未处理的异常情况及边界条件。
            3. 性能隐患(如N+1查询、内存泄漏风险)。
            请按以下格式输出报告。
            ## 审查报告
            ### Critical (必须修复)
            ### Warning (建议修复)
            ### Suggestion (可选优化)
          claude_args: >-
            --allowedTools "Read,Grep,Glob"
            --max-turns 10
            --model claude-sonnet-4-6

```

这段配置有以下3点值得特别说明。

1 两重触发模式

官方Action v1版本具备自动检测触发模式的能力。

- **Agent Mode（代理模式）**：当配置中提供了 `prompt` 参数时触发。每次PR创建或更新，Claude都会自动执行预设的审查任务。
- **Tag Mode（标签模式）**：当用户在PR评论中提及 `@claude` 时触发。此时Claude将像团队成员一样，针对具体提问进行交互式响应。

这两种模式可在同一个Workflow文件中共存，互不冲突。

2 参数透传机制

官方Action v1版本大幅简化了配置结构。所有Claude Code CLI的参数现统一通过 `claude_args` 字段传递。此前Beta版本中独立的 `max_turns`、`allowed_tools` 等字段已被废弃，不再支持。

3 并发控制策略

通过设置 `cancel-in-progress: true`，可实现高效的并发控制。当同一PR连续推送多次提交时，系统自动取消正在运行的旧任务，仅保留最新提交所触发的审查流程。这一机制能有效避免重复执行，从而节省API调用费用。

7.4.2 直接调用CLI

当你需要实现更复杂的逻辑，例如，根据审查结果决定是否阻塞PR合并、将审查报告发布为正式的PR评论而非普通评论，或针对不同类型的文件应用差异化的审查策略时，直接调用Claude CLI将有更大的灵活性。

```
name: Advanced AI Review

on:
  pull_request:
    types: [opened, synchronize]
    paths:
      - 'src/**'    # 仅在src目录发生变更时触发

concurrency:
  group: ${{ github.workflow }}-${{ github.event.pull_request.number }}
  cancel-in-progress: true

jobs:
  review:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write
```

steps:

- uses: actions/checkout@v4
with:
fetch-depth: 0
- uses: actions/setup-node@v4
with:
node-version: "20"
- name: Install Claude Code
run: npm install -g @anthropic-ai/claude-code
- name: Get changed files
id: changed
run: |
FILES=\$(git diff --name-only origin/\${{ github.base_ref }}...HEAD)
echo "files=\$(echo "\$FILES" | tr '\n' ' ')" >> \$GITHUB_OUTPUT
echo "count=\$(echo "\$FILES" | wc -l)" >> \$GITHUB_OUTPUT
- name: Run Claude Review
id: review
env:
ANTHROPIC_API_KEY: \${{ secrets.ANTHROPIC_API_KEY }}
CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC: "1"
run: |
claude -p "请审查以下PR变更文件：
\${{ steps.changed.outputs.files }}"

重点检查安全漏洞、逻辑错误及性能问题。
针对每个问题，请明确指出文件名和行号。" \
--output-format json \
--max-turns 10 \
--max-budget-usd 0.50 \
--model claude-sonnet-4-6 \
--allowedTools "Read,Grep,Glob" > review.json

提取审查结果与成本信息
RESULT=\$(jq -r '.result' review.json)
COST=\$(jq -r '.total_cost_usd' review.json)
IS_ERROR=\$(jq -r '.is_error' review.json)

echo "cost=\$COST" >> \$GITHUB_OUTPUT
echo "is_error=\$IS_ERROR" >> \$GITHUB_OUTPUT
echo "result<<EOF" >> \$GITHUB_OUTPUT
echo "\$RESULT" >> \$GITHUB_OUTPUT
echo "EOF" >> \$GITHUB_OUTPUT

```

- name: Post Review Comment
  uses: actions/github-script@v7
  with:
    script: |
      const cost = `${{ steps.review.outputs.cost }}`;
      const result = `${{ steps.review.outputs.result }}`;

      await github.rest.issues.createComment({
        issue_number: context.issue.number,
        owner: context.repo.owner,
        repo: context.repo.repo,
        body: `## Claude Code Review\n\n${result}\n\n---\n\nCost: ${cost}
| Automated by Claude Code`
      });

- name: Gate on critical issues
  run: |
    RESULT="${{ steps.review.outputs.result }}"
    if echo "$RESULT" | grep -qi "critical"; then
      echo "::error::Critical issues found"
      exit 1
    fi

```

上述配置虽然较官方Action复杂，但赋予了用户3项关键能力：**成本透明化**（每次审查的具体费用将直接显示在评论中）、**质量门禁**（当检测到Critical级别的严重问题时，Workflow会自动失败并阻塞PR合并）、**路径过滤**（通过paths限定，仅在src/目录发生变更时触发审查）。

此外，需要注意 `CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC` 环境变量。这是CI/CD流水线中的标准最佳实践，用于一次性禁用自动更新检查、错误上报及遥测数据发送等在自动化场景中非必要的功能。

7.5 实战二：多阶段CI管道

7.4节介绍的案例属于单阶段审查。在实际项目中，可能需要让Claude参与多个阶段，例如，先审查代码质量，再扫描安全漏洞，最后检查测试覆盖率。每个阶段往往具有不同的关注点、工具权限限制以及成本预算。

以下是一个多阶段AI流水线的配置示例。

```

name: Multi-Stage AI Pipeline

on:
  pull_request:
    types: [opened, synchronize]

```

jobs:

阶段1: 快速格式检查(使用低成本模型)

lint-check:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4
- name: Format Check

env:

ANTHROPIC_API_KEY: \${ secrets.ANTHROPIC_API_KEY }

CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC: "1"

run: |

```
npx @anthropic-ai/claude-code -p "检查src/下的代码是否符合项目编码规范" \  
  --model claude-haiku-4-5 \  
  --max-turns 3 \  
  --max-budget-usd 0.05 \  
  --allowedTools "Read,Grep,Glob" \  
  --output-format text
```

阶段2: 深度安全审查(使用高性能模型)

security-scan:

runs-on: ubuntu-latest

needs: lint-check # 仅在格式检查通过后执行

steps:

- uses: actions/checkout@v4
- name: Security Scan

env:

ANTHROPIC_API_KEY: \${ secrets.ANTHROPIC_API_KEY }

CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC: "1"

run: |

```
npx @anthropic-ai/claude-code -p "深度安全扫描，重点检查OWASP Top 10" \  
  --model claude-sonnet-4-6 \  
  --append-system-prompt "你是一名应用安全专家，请仅关注安全漏洞问题。" \  
  --max-turns 10 \  
  --max-budget-usd 1.00 \  
  --allowedTools "Read,Grep,Glob" \  
  --output-format json \  
  --json-schema '{"type":"object","properties":{"vulnerabilities":
```

```
{"type":"array","items":{"type":"object","properties":{"severity":
```

```
{"type":"string"},"description":{"type":"string"},"file":
```

```
{"type":"string"}}, "required":["severity","description"]}}',"required":
```

```
["vulnerabilities"]}' \  
  > security-report.json
```

阶段3: 测试覆盖分析

coverage-check:

```

runs-on: ubuntu-latest
needs: lint-check # 依赖于格式检查，但与安全检查并行
steps:
  - uses: actions/checkout@v4
  - name: Coverage Analysis
    env:
      ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
      CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC: "1"
    run: |
      npx @anthropic-ai/claude-code -p "分析哪些代码路径缺少测试覆盖" \
        --model claude-sonnet-4-6 \
        --max-turns 8 \
        --max-budget-usd 0.50 \
        --allowedTools "Read,Grep,Glob" \
        --output-format text

```

上述配置涉及3个设计要点。

- **模型分级：**针对轻量级任务（如格式检查），选用 Haiku（该模型响应速度快、成本低）；针对重量级任务（如安全扫描和测试覆盖分析），选用 Sonnet（利用其更强的推理能力进行深度分析）。根据不同任务的复杂度匹配相应的“算力配比”，实现效率与效果的最优平衡。
- **预算分级：**为简单任务设定严格上限（如0.05美元），为复杂任务预留充足空间（如1美元）。通过 `--max-budget-usd` 参数精确控制单次运行的最大支出，确保整体流水线成本可控且可预测。
- **并行执行：**在任务 `security-scan` 和 `coverage-check` 中均设置了 `needs: lint-check`，意味着它们必须等待第一阶段完成后才能启动。由于这两个任务之间没有相互依赖，GitHub Actions会自动将它们并行运行。

7.6 流式输出：实时监控长耗时任务

对于耗时较长的分析任务，`stream-json` 格式允许用户实时观测Claude的执行进度，而不需要等待工作全部完成后才查看最终结果。请看以下代码示例。

```

claude -p "分析整个src/目录的架构问题" \
  --output-format stream-json \
  --allowedTools "Read,Grep,Glob" | while read -r line; do
    TYPE=$(echo "$line" | jq -r '.type // ""')

    case "$TYPE" in
      "system")
        # 初始化阶段：输出模型信息
        echo "[INIT] 模型: $(echo "$line" | jq -r '.model // "unknown"')"
        ;;
      "assistant")

```

```

# 提取并输出Claude生成的文本内容
TEXT=$(echo "$line" | jq -r '.message.content.text // ""')
[ -n "$TEXT" ] && echo "[CLAUDE] $TEXT"
;;
"result")
# 任务结束：输出总轮数与费用
COST=$(echo "$line" | jq -r '.total_cost_usd // 0')
TURNS=$(echo "$line" | jq -r '.num_turns // 0')
echo "[DONE] 任务完成！共 $TURNS 轮，费用：\${$COST}"
;;
esac
done

```

stream-json 格式定义了一套完整的事件协议。除上述案例中涉及的 `system`（初始化）、`assistant`（助手回复）、`user`（用户输入）及 `result`（结果汇总）以外，还有两类特殊事件值得注意。

- `stream_event`（需要添加 `--include-partial-messages` 参数）：提供Claude输出的增量数据，精确到每一个Token级别的文本片段。适用于需要实现“打字机效果”的用户界面，但在CI自动化流程中通常不需要启用。
- `system/compact_boundary`：上下文压缩边界事件。当对话历史长度接近模型的上下文窗口限制时，Claude会自动触发机制压缩早期内容以腾出空间。在长耗时分析任务中，若检测到此事件，表明当前任务已消耗大量上下文资源，需要注意潜在的上下文丢失风险。

7.7 会话管理：跨步骤维持上下文

在Headless模式下，系统默认不持久化会话状态——每次调用均视为独立的全新上下文。然而，在涉及多轮交互的复杂场景中，往往需要在多次调用之间保持上下文的连续性。请看以下代码示例。

```

# 第一步：执行初始分析
# 调用Claude分析代码结构，并捕获完整的JSON输出
RESULT=$(claude -p "分析 src/ 的模块依赖关系" \
  --output-format json \
  --allowedTools "Read,Grep,Glob")

# 第二步：提取会话标识符
# 从返回结果中解析session_id，用于后续恢复会话
SESSION_ID=$(echo "$RESULT" | jq -r '.session_id')

# 第三步：基于上下文继续对话
# 使用 --resume 参数载入上一轮的会话状态，实现连续推理
claude -p "基于刚才的分析，指出循环依赖问题" \

```



```
--resume "$SESSION_ID" \  
--output-format text
```

`--resume` 参数的核心价值在于让Claude完整“继承”上一轮对话的上下文状态，包括已读取的文件内容、执行过的分析步骤以及得出的中间结论。相较于在新一轮Prompt中手动复述上下文信息，这种方式不仅显著提升了交互效率，更确保了模型推理的准确性与连贯性。

除基础的会话恢复以外，CLI工具还提供了一系列高级参数以应对复杂的交互场景。请看以下代码示例。

```
# 自动续接：继续当前目录下的最近一次会话  
# 不需要手动查找session_id，系统自动匹配并更新  
claude -p "继续刚才的分析" --continue  
  
# 会话分支：基于现有会话创建新分支  
# 保留原始会话不变，从指定节点衍生出新的探索路径  
claude -p "如果改用微服务架构呢？" --resume "$SESSION_ID" --fork-session  
  
# 禁用持久化：强制无状态模式  
# 适用于完全隔离的CI/CD流水线，确保不留下任何会话历史  
claude -p "一次性分析" --no-session-persistence
```

`--fork-session` 是一个极具实用价值的参数，其应用逻辑类似于版本控制系统中的Git分支机制。它从指定会话的某个时间点“分叉”出一个全新的会话实例。原始会话（主干）保持原样，不受任何影响；新会话（分支）则从该分叉点开始独立演进。这一特性完美契合“假设性分析”的需求。例如，在完成同一份代码库的基础分析后，开发者可以通过该参数创建多个分支，分别探讨不同的重构方向。

7.8 CI环境配置：生产级清单

在真实的CI/CD环境中部署Claude Code，除核心参数 `-p` 和 `--allowedTools` 以外，还需要设置一系列环境变量。以下是咖哥整理的“CI环境配置清单”。

7.8.1 必须设置的环境变量

必须设置的环境变量如下。

```
# 认证  
ANTHROPIC_API_KEY=sk-ant-... # API密钥  
  
# 功能禁用  
# 方案一：使用聚合变量(推荐，可一次性禁用多项非必要功能)  
CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC=1 # 禁用自动更新、遥测及错误上报
```

方案二：分别禁用(如需更细粒度的控制)

DISABLE_AUTOUPDATER=1

DISABLE_TELEMETRY=1

DISABLE_ERROR_REPORTING=1

禁用自动更新(CI环境中必需)

禁用遥测

禁用错误上报

DISABLE_AUTOUPDATER 是CI/CD环境中至关重要的环境变量。如果未设置该变量，Claude Code 可能在执行过程中尝试自动更新，从而引发版本不一致或服务意外中断的风险。相比之下，CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC 则是一个便捷的“总开关”，能够一次性禁用所有非必要的后台流量（包括自动更新、遥测及错误上报），简化配置流程。

7.8.2 性能调优参数

涉及性能调优的参数如下。

超时控制

Bash 命令默认超时时间（单位：毫秒）

BASH_DEFAULT_TIMEOUT_MS=120000

对应2min

Bash 命令最大允许超时时间（单位：毫秒）

BASH_MAX_TIMEOUT_MS=600000

对应10min

输出控制

CLAUDE_CODE_MAX_OUTPUT_TOKENS=32000

32 000, 上限64 000)

模型单次响应最大Token数(默认值

MAX_MCP_OUTPUT_TOKENS=25000

MCP工具响应最大Token数

上下文管理

CLAUDE_CODE_AUTOCOMPACT_PCT_OVERRIDE=50

代表上下文使用百分比；默认值50%)

自动压缩触发阈值(取值范围1~100,

7.8.3 GitHub Actions中的完整配置模板

以下是在GitHub Actions工作流中部署Claude Code的完整配置片段。

- name: Run Claude Analysis

env:

认证密钥(从仓库Secrets中获取)

ANTHROPIC_API_KEY: \${ secrets.ANTHROPIC_API_KEY }

性能与环境控制

CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC: "1" # 禁用非必要后台流量

BASH_DEFAULT_TIMEOUT_MS: "300000" # Bash命令默认超时: 5 min

run: |

npx @anthropic-ai/claude-code -p "\$PROMPT" \

```
--output-format json \  
--max-turns 10 \  
--max-budget-usd 1.00 \  
--allowedTools "Read,Grep,Glob" \  
--model claude-sonnet-4-6 \  
--fallback-model haiku \  
--append-system-prompt "遵循项目根目录CLAUDE.md中的审查规范。"
```

7.8.4 MCP服务器在CI中的配置

如果CI任务需要借助MCP服务器扩展能力（如数据库查询、外部API调用等），可通过 `--mcp-config` 参数加载专用配置文件，并配合 `--strict-mcp-config` 确保环境隔离。请看以下代码示例。

```
npx @anthropic-ai/claude-code \  
  -p "分析数据库Schema的优化空间" \  
  --mcp-config ./ci-mcp-config.json \  
  --strict-mcp-config \  
  --allowedTools "Read,Grep,Glob,mcp__database__query" \  
  --max-turns 10
```

`--strict-mcp-config` 参数表示启用严格模式，强制Claude仅使用 `--mcp-config` 指定的配置文件。忽略用户主目录和项目根目录中的其他MCP配置。这在CI环境中很重要——可以防止开发者本地残留的MCP配置被意外加载。

7.9 跨平台CI/CD集成

尽管GitHub Actions拥有官方Action支持，但Claude Code的Headless模式本质上是与平台无关的。只要运行环境支持Node.js，即可在任何CI/CD平台上部署。

7.9.1 GitLab CI/CD配置

以下配置展示了如何在GitLab Runner中执行代码审查任务，并将结果结构化输出。

```
# .gitlab-ci.yml  
claude-review:  
  image: node:20  
  variables:  
    ANTHROPIC_API_KEY: $ANTHROPIC_API_KEY  
    CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC: "1"  
  script:  
    - npm install -g @anthropic-ai/claude-code  
    - |  
      claude -p "审查MR的代码变更，重点关注潜在的安全漏洞和性能问题" \  
    - |
```

```

--output-format json \
--max-turns 10 \
--max-budget-usd 0.50 \
--allowedTools "Read,Grep,Glob" > review.json
- cat review.json | jq -r '.result'
rules:
- if: $CI_PIPELINE_SOURCE == "merge_request_event"

```

值得注意的是，Anthropic与GitLab的合作正在深化。目前GitLab已推出官方AI功能（Beta阶段），引入了 `AI_FLOW_INPUT` 和 `AI_FLOW_CONTEXT` 等专用变量，不需要手动拼接Prompt上下文；通过 `gitlab-mcp-server` 直接集成，允许AI模型安全地访问GitLab API。

7.9.2 Jenkins Pipeline集成配置

在Jenkins中，可以通过Declarative Pipeline（声明式流水线）轻松集成Claude Code。以下配置展示了如何安全地管理凭据、执行Headless模式下的代码审查，并解析结构化输出。

```

pipeline {
    agent any
    environment {
        ANTHROPIC_API_KEY = credentials('anthropic-api-key')
        CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC = '1'
    }
    stages {
        stage('AI Code Review') {
            steps {
                sh 'npm install -g @anthropic-ai/claude-code'
                sh '''
                    claude -p "审查最近提交的代码变更，重点关注逻辑错误和代码规范" \
                        --output-format json \
                        --max-turns 10 \
                        --max-budget-usd 0.50 \
                        --allowedTools "Read,Grep,Glob" > review.json
                '''
                script {
                    def review = readJSON file: 'review.json'
                    echo "审查结果: ${review.result}"
                    echo "费用: \${review.total_cost_usd}"
                }
            }
        }
    }
}

```

7.9.3 本地自动化脚本

除CI/CD平台以外，Claude Code的Headless模式也非常适合封装成本地脚本，供团队成员在提交代码前进行自检，或作为本地自动化工具链的一部分。

以下是一个增强版的 `review.sh` 脚本，具备环境检查、动态文件筛选、结构化报告生成及错误处理能力。

```
#!/bin/bash
# review.sh - 团队共享的本地代码审查脚本
set -e

TARGET=${1:-.}

# 前置检查
[ -z "$ANTHROPIC_API_KEY" ] && echo "Error: ANTHROPIC_API_KEY not set" && exit 1
command -v claude &> /dev/null || { echo "Error: Claude Code not installed"; exit 1; }

echo "Starting review for: $TARGET"

# 构建文件列表
if [ -d "$TARGET" ]; then
    FILES=$(find "$TARGET" -type f \( -name "*.ts" -o -name "*.js" -o -name "*.py" \) | head -20)
else
    FILES="$TARGET"
fi

# 运行审查
RESULT=$(claude -p "审查以下文件的代码质量和安全问题：
$FILES

输出格式：使用Markdown格式，每个问题必须指明文件名和行号。" \
    --output-format text \
    --max-turns 15 \
    --max-budget-usd 0.50 \
    --allowedTools Read,Grep,Glob)

# 保存报告
REPORT="review-$(date +%Y%m%d-%H%M%S).md"
echo -e "# Code Review Report\n\n**Date**: $(date)\n\n**Target**: $TARGET\n\n---\n\n$n$RESULT" > "$REPORT"
echo "Report saved: $REPORT"
```

7.10 安全原则与最佳实践

“在CI/CD中运行Agent是一件需要认真对待的安全事项。”咖哥的语气严肃了一些。

这绝非杞人忧天。一个配置不当的Headless模式任务可能引发以下严重风险。

- **机密泄露**：读取并输出 `.env`、`config.json` 或硬编码的Secrets文件。
- **破坏性操作**：执行恶意的Shell命令，破坏构建环境或覆盖代码。
- **费用失控**：陷入死循环或被诱导生成大量无意义内容，消耗大量API调用费用。
- **内容污染**：在PR/MR评论中输出幻觉内容、不当言论甚至被注入攻击载荷。

防御策略必须从以下5个层面构建纵深防御体系。

7.10.1 最小权限原则

在实际应用场景中，可采取以下措施。

```
# 代码审查任务：只读
--allowedTools "Read,Grep,Glob"

# 格式修复任务：读写
--allowedTools "Read,Grep,Glob>Edit,Write"

# Git操作任务：细粒度Bash白名单
--allowedTools "Read,Grep,Glob,Bash(git diff *),Bash(git log *)"
```

严禁授予Claude无限制的Bash执行权限。即便在必须使用Shell能力的场景中，也必须通过模式匹配严格限制可执行命令的范围。

7.10.2 Secrets管理

针对Secrets管理，这里通过正确与错误的做法对比说明。

```
# 错误做法：硬编码密钥(密钥将直接明文出现在 Actions 运行日志中)
env:
  ANTHROPIC_API_KEY: "sk-ant-xxx"

# 正确做法：引用仓库密钥
env:
  ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
```

GitHub Actions会自动在日志中对secrets引用的值进行遮蔽（显示为 ***）。

尽管GitHub具备日志遮蔽机制，但若在Prompt中指示模型“列出所有环境变量”或“打印配置详情”，模型可能会在输出文本中复述密钥值。这种由模型生成的文本无法被自动遮蔽机制捕获，从而导致密钥泄露。

严禁在Prompt中要求模型访问、读取或输出任何环境变量及密钥值。

7.10.3 容器隔离

以下展示了用于构建高度安全的“沙箱”环境以运行代码审查任务的GitHub Actions workflow配置片段。

```
jobs:
  review:
    runs-on: ubuntu-latest
    container:
      image: node:20
      options: --read-only --tmpfs /tmp --network none
```

`--read-only`：将文件系统设为只读，防止写入操作（配合使用 `--tmpfs /tmp` 为临时文件提供可写空间）。

`--network none`：在不需要外部访问的审查任务中完全切断网络连接。在此模式下，Claude仅能读取代码仓库内的文件，无法与外界通信。

7.10.4 成本防护

以下展示了GitHub Actions workflow中用于优化资源使用并降低运行成本的配置片段。

```
# 路径过滤：仅在src目录变更时触发
on:
  pull_request:
    paths:
      - 'src/**'
      - '!src/**/*.test.*'    # 排除测试文件

# 并发限制：同一PR仅保留最新一次运行，自动取消中间版本
concurrency:
  group: claude-${{ github.event.pull_request.number }}
  cancel-in-progress: true
```

路径过滤与并发控制是成本优化的首道防线。相比于限制单次运行的资源消耗，从源头避免不必要的触发能更有效地降低总成本。

7.10.5 审计日志

以下展示了执行一个AI任务并将包含完整审计信息的运行日志持久化保存的配置片段。

```
# JSON输出格式天然包含完整的审计信息
claude -p "任务" --output-format json | tee -a /var/log/claude-audit.jsonl
```

每次Headless模式调用的JSON输出均自动包含 `session_id`、`total_cost_usd`、`num_turns` 及 `usage` 等关键审计字段。为严格遵循合规要求，建议将这些日志实时推送至集中式日志系统（如 ELK、Datadog等），以实现长期归档与深度分析。

咖哥发言

以上五层防御绝非“选做题”，而是生产环境中的“必答题”：最小权限原则杜绝Claude“做不该做的事”，Secrets管理严防凭据泄露，容器隔离阻断逃逸风险，成本防护避免账单失控，审计日志确保全程可追溯。缺失任何一层，都意味着安全防线出现了致命缺口。

7.11 从CLI到Agent SDK：Headless模式的编程接口

前面介绍的示例均是通过CLI来调用Claude Code，但在实际工程实践中，Claude Code还提供了Agent SDK（支持TypeScript和Python），允许开发者直接使用编程语言而非命令行参数来控制Headless模式执行流程。

以下是TypeScript Agent SDK 和 Python Agent SDK示例。

```
// TypeScript Agent SDK示例
import { query } from "@anthropic-ai/claude-agent-sdk";

async function reviewPR(changedFiles: string[]) {
  let sessionId: string | undefined;
  let result: string = "";

  for await (const message of query({
    prompt: `审查以下文件的代码变更: ${changedFiles.join(", ")}`,
    options: {
      allowedTools: ["Read", "Grep", "Glob"],
      maxTurns: 10,
      maxBudgetUsd: 0.50,
      model: "claude-sonnet-4-6",
      outputFormat: { type: "json_schema", schema: reviewSchema },
    },
  })) {
    if (message.type === "system" && message.subtype === "init") {
      sessionId = message.session_id;
    }
    if (message.type === "result" && message.structured_output) {
      result = message.structured_output;
    }
  }
}
```



```

    return { sessionId, result };
}
# Python Agent SDK示例
from claude_agent_sdk import query, ClaudeAgentOptions, ResultMessage

async def review_pr(changed_files: list[str]):
    options = ClaudeAgentOptions(
        allowed_tools=["Read", "Grep", "Glob"],
        max_turns=10,
        max_budget_usd=0.50,
        model="claude-sonnet-4-6",
    )

    async for message in query(
        prompt=f"审查以下文件: {' '.join(changed_files)}",
        options=options,
    ):
        if isinstance(message, ResultMessage):
            return message.result

```

CLI与Agent SDK的关系，恰如curl与HTTP客户端库之别：CLI胜在轻量便捷，适用于简单的自动化脚本及CI/CD流水线；而Agent SDK则专为复杂逻辑场景而生——无论是基于上一轮审查结果动态调整后续Prompt，还是在多次Claude调用时进行精细的流程编排，抑或是要将Claude深度集成至大型应用系统中，Agent SDK都能提供开发者所需的编程灵活性。在第8章中，我们将深入剖析Agent SDK的具体用法。

7.12 渐进式落地策略

“说了这么多技术细节，”小冰问，“如果我们团队想引入Headless模式进行PR审查，该如何分步推进？”

咖哥建议采取四阶段渐进式落地路径。

阶段一：观察者模式。 Claude仅执行只读分析，输出审查报告但不阻塞任何流程。审查报告以评论形式发布在PR中，供团队成员参考或忽略。这个阶段的核心目标是建立信任——让团队直观感受Claude的审查质量，同时识别其局限性（如易误报场景），据此迭代优化Prompt与参数配置。

阶段二：顾问模式。 在积累充足的正面反馈后，将Claude的审查结果纳入CI状态监控。发现严重(Critical)问题时，将CI状态标记为“警告”(黄色)，但仍不阻塞合并。人工审查员需要结合Claude的报告做出最终判断。这个阶段的核心目标是人机协作。

阶段三：门禁模式。 Claude的审查成为代码合并的必要条件之一。一旦检测到严重安全问题，CI状态立即变红并阻塞合并。但在此阶段必须配套“逃生通道”，允许拥有特定权限的人员手动覆

盖Claude的阻塞决定（类似于跳过不稳定的测试用例）。

阶段四：主动修复模式。 Claude不仅能发现问题，还能自动修复格式错误、补充缺失的异常处理逻辑或生成单元测试。此阶段需要开放Write工具权限，但所有修复必须以新Commit的形式提交，严禁直接修改PR作者的原始代码。

“切忌一步到位，”咖哥强调，“每个阶段至少运行2到4周，密切监控误报率、成本趋势及团队接受度。AI辅助审查的核心价值不在于‘完美替代人工’，而在于‘让人类审查员将有限的精力聚焦于最具价值的决策上’。”

本章小结

Headless模式推动了Claude Code的角色蜕变——从局限于终端交互的智能助手，进化为可无缝嵌入各类工程流程的可编程组件。这一转变由4个维度的参数定义。

- `-p`（入口）：将Claude Code的运行模式从交互式会话切换为非交互式执行，奠定自动化基础。
- `--output-format`（数据接口）：规范输出结构，确保下游系统能高效解析与调用审查结果。
- `--allowedTools`（安全边界）：严格限定工具权限，确保自动化环境中的操作合规、可控。
- `--max-turns` 与 `--max-budget-usd`（成本护栏）：设定执行轮次与预算上限，防止无人值守场景下的资源失控。

在工程架构层面，Headless模式的巨大价值在于赋予Claude Code“CI/CD生态平等公民”的身份。如同单元测试、静态分析或容器镜像构建，AI代码审查正式成为流水线中的标准环节：它需要具备明确的输入定义、确定性的输出格式、可量化的成本预算，以及可纳入版本控制的配置文件。

在落地策略上，建议遵循“观察者模式”起步，逐步演进至“门禁模式”的路径。这一渐进过程旨在为团队预留充足的时间以建立信任、积累实战数据并优化配置。切记不可急于求成——AI辅助审查的核心价值并非追求对人工的“完美替代”，而是通过智能过滤，让人类审查员将宝贵的精力聚焦于最具关键价值的决策环节。

在第8章中，我们将深入从命令行参数迈向编程接口(Agent SDK)的世界。你将掌握如何使用TypeScript或Python代码取代烦琐的CLI参数，灵活控制Claude的行为逻辑，从而构建真正定制化、高度集成的AI应用系统。

思考题

1. 将Claude Code的PR审查集成至现有CI/CD流水线，不仅需要攻克技术配置难关，更需要妥善解决团队协作流程中的核心议题：如何确立审查结果的权威性？如何厘清其与现有Linter工具的职能边界？如何构建高效的误报处理机制？

2. `--max-turns 5` 与 `--max-budget-usd 0.50` 分别在何种场景下会率先触发限制？在设计生产环境的Headless模式任务时，你应优先关注哪一指标？请阐述理由。
3. 请明确你的团队所使用的CI平台（如GitHub Actions、GitLab CI、Jenkins或其他）。基于此，请为项目量身定制一套Claude Code集成方案，涵盖触发条件、工具权限、成本预算及安全策略，并输出完整的配置文件。